

교통사고 심각도 ML 프로젝트 포트폴리오 요약

프로젝트 개요

Kaggle RoadSense 교통사고 데이터셋 사고/도로/차량/인적 정보 총 4개의 데이터 셋을 활용해 사고 심각도 - Gravity를 예측하는 모델을 프로젝트를 진행하였으며 평가지표는 대회 기준인 macro F1을 사용하였다.

1. 1차 전처리 [주요 처리 : 라벨 통일]

Train 데이터는 날짜·시간·좌석 위치·차종 등 컬럼 표기 형식이 통일되어 있지 않다.
같은 의미이지만 라벨의 형식이 제각각으로 섞여 있거나 날짜형식이 약어가 있거나 년,월,일 위치가 제각각이었다.
반면 Test 데이터는 이미 정리된 형식이었다.

해결

- Test 데이터의 표기 형식을 기준으로 삼아 Train 데이터를 정규표현식으로 통일하였다.
- 다국어 및 오타가 섞인 범주형 값들을 매핑 테이블로 표준화 하였다.
- 단순 중복 제거가 아닌 병합 과정에서 정보 손실이 없도록 처리하였다.
 - places 테이블에서 중복행이 서로 다른 결측 패턴을 보완하는 관계임을 발견하여 오염값인 Modified를 NaN으로 치환하여 처리하였다.

[데이터 통일 예제]

before		After	
<pre>[169]: # Date 데이터 조화 accidents_train["Date"].head(10)</pre>		<pre>[180]: # 최종 변환 확인 accidents_train["Date"].head(10)</pre>	
0	24/01/2018	0	2018-01-24
1	Feb 12 2018	1	2018-02-12
2	04-Mar-18	2	2018-04-03
3	May 05 2018	3	2018-05-05
5	2018-11-30 00:00:00	5	2018-11-30
6	Feb 18 2018	6	2018-02-18
8	28/05/2018	8	2018-05-28
9	May 31 2018	9	2018-05-31
10	15/06/2018	10	2018-06-15
11	2018-07-19 00:00:00	11	2018-07-19
Name: Date, dtype: str		Name: Date, dtype: str	

[Modified 패턴 발견]

```
•[62]: #AccidentId 중복행 확인/해보기 (AccidentId를 기준으로 오름차순 정렬)
places_train[places_train["AccidentId"].duplicated(keep=False)].sort_values("AccidentId").head(20)
```

	AccidentId	RoadType	RoadNumber	RoadSecNumber	RoadLetter	Circulation	LaneNumber	SpecialLane	Slope	RoadMarkerId	RoadMarkerDistance
16	201800000019	Departmental	145	NaN	NaN	TwoWay	2.0	0	Flat	NaN	NaN
53801	201800000019	Departmental	Modified	NaN	NaN	TwoWay	2.0	0	Flat	NaN	NaN
52350	201800000020	Modified	NaN	NaN	NaN	TwoWay	2.0	0	Flat	NaN	NaN
17	201800000020	Communal	NaN	NaN	NaN	TwoWay	2.0	0	Flat	NaN	NaN

2. EDA 및 2차 전처리 [주요 처리 : 범주 재해석]

일부 범주형 변수는 값 하나에 사고 수심만건 대 수심 건의 극단적인 라벨의 분포로 그대로 모델에 적용하게 된다면 소수 범주에서 안정적인 패턴을 학습하기 어렵다고 판단하였다. 또한, 결측치 처리 기준이 컬럼마다 다르기 때문에 일괄 처리로는 정보 손실이 있으며 보다 결측치에 대한 배경적인 의미를 최대한 살려 재해석하려고 노력하였다.

해결

- 표본 수가 적은 세부 범주는 사고 위험도를 기준으로 유사한 값끼리 묶어 재정의 하였다.
 - 예를 들어 차량 유형을 그룹화하거나 회전/역주행 등 Maneuver를 위험도에 따라 분류하였다.
- 컬럼 성격에 맞춰 결측치 처리 기준을 다르게 적용하였다.
 - 범주 특성상 '알 수 없음'은 Unknown 라벨로 애초에 관측되지 않은 값은 'Missing'으로 분류

추가

다중공선성 검토

- Cramér's V로 범주형 변수 간 연관성을 정량적으로 확인하였다.
- Age와 AgeGroup처럼 완전 중복인 변수는 파생 변수만 남기고 원본을 제거하였다.
- 나머지 변수들은 도메인상 자연스러운 연관으로 판단하여 유지하였다.

[처리 예제]



[다중공선성 정리표(요약)]

데이터셋	Col1	Col2	Cramér's V	비고
users	Age	AgeGroup	1.000	완전 중복(파생 컬럼) → Age 제외, AgeGroup만 사용
users	PedestrianCompany	RoadUserType	0.574	보행자 변수군 상호 연관 매우 높음
users	SafetyDevice	SafetyDeviceUsed	0.542	보유여부·착용여부 개념적 유사, 통합 검토
users	Gender	Age	0.446	
accidents	Light	InAgglomeration	0.387	야간/시가지 여부 연관
places	RoadType	Circulation	0.352	도로유형 관련 변수군 간 연관 높음
accidents	InAgglomeration	IntersectionType	0.280	
vehicles	FixedObstacle	MobileObstacle	0.255	

3. 피처 엔지니어링 [주요 처리 : 집계 데이터 생성 및 모든 데이터셋 merge]

사고 하나당 AccidentId로 이루어져있는 컬럼이지만 Users, Vehicles 테이블은 여러명의 사람과 차량이 연결되는 1:N 구조를 가지고있어 사고 단위 1행인 컬럼들과는 결합이 어려웠다. 이 문제를 해결하기 위해 1:N 구조의 데이터 셋을 집계 데이터로 생성하고 1:1 구조로 변환하여 merge하는 작업을 진행하였다.

해결

- Users, Vehicles를 AccidentId기준으로 집계하여 1:1 구조로 변환하였다.
 - 변환된 컬럼은 [인원수, 성별 수, 운전자 수, 보행자 수, 차종별 원-핫 카운트 등 사고 단위 요약 피처가 생성되었다.
- accidents + places + users + vehicles를 AccidentId로 순차 merge하여 사고 단위 학습 데이터셋 구성하였다.
- 트리 기반 모델 학습을 위해 Accidents와 Places에 남아있는 변수형 변수를 원-핫 인코딩 처리하여 최종 194개 피처로 확장하였다.

4. 모델링 및 결과 [주요 처리 : 모델 실험]

[모델 선정]

- RandomForest
- XGBoost

[모델 선정 이유]

다수의 범주형 변수로 구성된 데이터 특성을 고려하여 트리 기반 모델을 실험 모델로 선정하였다. 원-핫 인코딩으로 194개 피처로 확장된 희소 및 이진화 구조에서도 트리 기반 모델은 별도 스케일링 없이 안정적으로 분기 기준으로 학습할 수 있다는 점에서 적합하다고 판단하였다.

[진행 과정]

1. RandomForest 교차검증 및 임계값 조절, 모델 학습, 성능 평가
 - StratifiedKFold 교차검증 결과 기본 임계값(0.5) 기준 precision이 0.43, recall이 0.10으로 양성 클래스를 거의 찾아내지 못하였다.
 - OOF(out-of-fold) 예측확률로 precision-recall curve를 시각화하여 F1이 최대가 되는 임계값을 탐색하였다.
 - 전체 Train 데이터로 재학습 후 held-out Test로 성능 평가시 F1 스코어가 0.3141이 최종적으로 나왔다.
2. XGBoost 교차검증 및 임계값 조절, 모델 학습·성능평가
 - 클래스 불균형 비율을 반영하여 scale_pos_weight를 계산하여 16.98을 적용하였다.
 - RandomForest와 동일한 방식으로 교차검증하였고 탐색결과 임계값 0.3050을 확인하였다.
 - 결과적으로 F1 스코어는 0.2747로 RandomForest보다 낮은 성능을 기록하였다.
 - 이는 원-핫 인코딩된 희소 피처가 심한 클래스 불균형 상황에서는 boosting 계열보다는 배깅 기반 모델이 상대적으로 안정적으로 작동했을 가능성이 있다고 해석하였다.
3. RandomForest 하이퍼파라미터 튜닝 및 추가 성능평가
 - RandomizedSearchCV로 튜닝을 진행하였다.
 - 튜닝 모델도 동일하게 임계값을 구한 후 Test 데이터로 평가를 진행시 F1 점수가 0.3128이 나왔다.
 - 이는 기본 모델 대비하여 성능이 개선되지 않음을 확인하였다.

[성능 결과표]

모델	threshold	accuracy	precision	recall	F1
RandomForest (기본)	0.2767	0.89	0.24	0.45	0.3141
RandomForest (튜닝)	0.5672	0.89	0.24	0.45	0.3128
XGBoost (기본)	0.3050	0.87	0.20	0.45	0.2747

최종해석

- 양성 클래스 표본수가 532건으로 매우 적은 데이터였으며 기본 모델과 튜닝 모델의 F1 차이는 통계적으로 유의미하다기보다 오차 범위 내 결과로 판단하였다.
- 하이퍼파라미터 튜닝이 성능을 극적으로 개선하지 못했다는 점은 문제의 근본 원인이 모델 구조가 아니라 데이터 자체의 클래스 불균형과 양성 표본 부족에 있을 수 있다고 판단하였다.
- 최종 F1 점수는 Kaggle 리더보드 상위권 팀 점수와 유사한 수준으로 성능 한계가 모델 선택이나 튜닝 부족이 아닌 데이터 자체의 근본적인 한계에서 기인함을 뒷받침한다.
- 추가적으로 치명/비치명 여부만으로 결과를 본다면 해도 모델이 대부분의 위험을 감지해내지 못하고 있어 감지 성능을 더 끌어올린다 해도 모델을 신뢰할 수 있는 수준이라는 생각이 들지 않으며 데이터의 원천적인 문제를 먼저 해결해야 한다고 생각하였다.